
Tromperie du cache Web

Quand une page privée devient une « ressource statique »

Gana Faye

Sécurité des applications web | 19 juillet 2026

RISQUE ÉLEVÉ

À la fin, vous saurez :

- ▶ expliquer le rôle d'un cache web ;
- ▶ reconnaître une divergence d'interprétation d'URL ;
- ▶ distinguer *Web Cache Deception* et *Poisoning* ;
- ▶ suivre l'exploitation réalisée avec Burp Suite ;
- ▶ évaluer l'impact et proposer des corrections vérifiables.

Fil conducteur

Une URL unique est interprétée de deux façons :

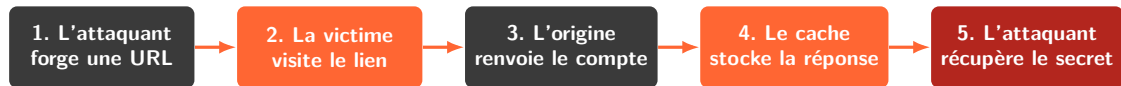
ORIGINE : PRIVÉ

≠

CACHE : PUBLIC

Démonstration exclusivement réalisée dans le laboratoire autorisé de PortSwigger Web Security Academy.

Le scénario en une minute



URL utilisée dans le laboratoire

`/my-account/wcd.js`

1

clé API exposée

30 s

durée de cache

2

interprétations de l'URL

PRÉREQUIS

Définitions et périmètre

Quel système est réellement ciblé ?

Cible d'une attaque contre le cache web

Le *Web Cache Poisoning* et la *Web Cache Deception* visent l'infrastructure qui distribue un site ou une application web : serveur HTTP, reverse proxy, CDN et mécanisme de cache.

- ▶ La vulnérabilité se situe côté serveur ou intermédiaire réseau.
- ▶ Le navigateur sert à envoyer les requêtes HTTP.
- ▶ L'attaque exploite une mauvaise politique de stockage des réponses.

Et WhatsApp Web ?

WhatsApp Web s'affiche dans un navigateur, mais sa messagerie repose principalement sur des communications synchronisées et en temps réel, avec des mécanismes tels que les connexions persistantes et le chiffrement de bout en bout.

Cela ne signifie pas qu'il est « hors du Web » : ses pages et ressources HTTP peuvent utiliser des caches. En revanche, le contenu temps réel des conversations ne fonctionne pas comme une simple page HTML mise en cache.

Architecture minimale d'une application web



Concept	Définition
Serveur d'origine	Serveur responsable de produire la réponse officielle de l'application.
Reverse proxy	Intermédiaire placé devant l'origine ; il reçoit et transmet les requêtes.
CDN	Réseau de serveurs distribués rapprochant les contenus des utilisateurs.
Cache partagé	Stockage intermédiaire pouvant servir une même réponse à plusieurs clients.

HTTP : requête, réponse et en-têtes

Requête HTTP

Message envoyé par le client pour demander une ressource ou effectuer une action.

- ▶ **Méthode** : GET, POST, etc.
- ▶ **Chemin** : ressource demandée.
- ▶ **En-têtes** : métadonnées de la requête.
- ▶ **Corps** : données facultatives envoyées au serveur.

Réponse HTTP

Message retourné avec un statut, des en-têtes et éventuellement un contenu.

- ▶ **Statut** : 200, 302, 404, etc.
- ▶ **Content-Type** : format du contenu.
- ▶ **Cache-Control** : règles de stockage.
- ▶ **X-Cache** : indicateur ajouté par certains caches.



```
https://lab.example.net/my-account/wcd.js?lang=fr#profil
```

Élément	Exemple	Rôle
Schéma	https	Protocole utilisé pour la communication sécurisée.
Hôte	lab.example.net	Nom du serveur destinataire.
Chemin	/my-account/wcd.js	Ressource interprétée par le cache et l'origine.
Extension	.js	Apparence de fichier JavaScript ; elle peut déclencher une règle de cache.
Paramètre	?lang=fr	Valeur transmise au serveur ; parfois incluse dans la clé de cache.
Fragment	#profil	Partie traitée par le navigateur, normalement non envoyée au serveur.

ORIGINE : ROUTE DE COMPTE / CACHE : EXTENSION .JS

Concept	Définition
Route / endpoint	Chemin reconnu par l'application et associé à une fonction, par exemple <code>/my-account</code> .
Routage	Mécanisme qui choisit le code à exécuter selon la méthode HTTP et le chemin.
Suffixe de chemin	Segment ajouté après la route attendue, comme <code>/abc.js</code> .
Routage permissif	Comportement acceptant un suffixe inattendu au lieu de retourner une erreur.
URL canonique	Forme unique et officielle d'une ressource. Les variantes doivent être rejetées ou redirigées sûrement.
Normalisation	Décodage et simplification du chemin avant son interprétation.
Divergence	Deux composants normalisent ou classent différemment la même URL.

`/my-account/abc.js`

→

ORIGINE : `/my-account`

≠

CACHE : FICHIER `.JS`

Contenu statique

Fichier généralement identique pour tous les visiteurs.

- ▶ images, polices, CSS, JavaScript ;
- ▶ souvent stockable longtemps ;
- ▶ fréquemment servi par un CDN.

PARTAGE HABITUELLEMENT SÛR

Contenu dynamique

Réponse construite selon l'utilisateur, la session ou les données courantes.

- ▶ compte, panier, messagerie ;
- ▶ peut contenir des secrets ;
- ▶ doit appliquer une politique de cache stricte.

PRIVÉ DANS CE LABORATOIRE

La tromperie consiste à donner à un contenu dynamique l'apparence d'un fichier statique.

Concept	Définition	Risque dans le scénario
Authentification	Vérification de l'identité d'un utilisateur.	Accès à la page privée.
Session	État temporaire reliant plusieurs requêtes au même utilisateur connecté.	La réponse dépend de carlos ou wiener.
Cookie de session	Petite valeur envoyée automatiquement par le navigateur pour identifier la session.	La victime l'envoie en visitant l'URL forgée.
API	Interface permettant à des logiciels d'échanger des données ou des commandes.	Peut donner accès à des fonctions sensibles.
Clé API	Secret utilisé pour autoriser un client auprès d'une API.	Sa divulgation peut permettre une usurpation.

Important

Le cache ne vole pas directement le cookie : il stocke la réponse personnalisée produite grâce à ce cookie.

Terme	Définition
Clé de cache	Identifiant calculé à partir de l'URL et d'autres éléments pour retrouver une réponse stockée.
Entrée de cache	Association entre une clé et une copie de réponse.
MISS	La clé n'existe pas ou a expiré ; le cache interroge l'origine.
HIT	La clé existe ; le cache renvoie directement la copie mémorisée.
TTL / max-age	Durée de validité d'une entrée avant son expiration.
Cache partagé	Cache susceptible de répondre à plusieurs utilisateurs.
Purge	Suppression forcée d'une ou plusieurs entrées.
Normalisation	Transformation d'une URL vers une représentation canonique avant traitement.

MISS : CRÉATION

 →

HIT : RÉUTILISATION

Directives et en-têtes liés au cache

En-tête / directive	Signification
Cache-Control: max-age=30	La réponse peut rester fraîche pendant 30 secondes.
Cache-Control: public	Un cache partagé est autorisé à stocker la réponse.
Cache-Control: private	La réponse est destinée à un seul utilisateur et ne doit pas être stockée par un cache partagé.
Cache-Control: no-store	Aucun cache ne doit conserver la requête ou la réponse.
Pragma: no-cache	Directive historique surtout utilisée pour la compatibilité HTTP/1.0.
X-Cache: miss	Le cache n'a pas fourni une copie existante.
X-Cache: hit	La réponse a été servie depuis une entrée de cache.
Cookie	Transporte notamment l'identifiant de session du navigateur.
Authorization	Transporte des informations d'autorisation, souvent un jeton.

PAGE PUBLIQUE : POLITIQUE EXPLICITE

PAGE PRIVÉE : NO-STORE, PRIVATE

HTTP classique

Le client envoie une requête, puis le serveur retourne une réponse.

- ▶ adapté aux pages et fichiers ;
- ▶ réponses souvent mises en cache ;
- ▶ interaction généralement ponctuelle.

REQUÊTE



RÉPONSE

Connexion temps réel

Une connexion persistante permet des échanges continus dans les deux sens.

- ▶ WebSocket est un exemple courant ;
- ▶ utile pour chat, notifications et collaboration ;
- ▶ les messages ne sont pas des pages HTTP ordinaires à mettre en cache.

CLIENT



SERVEUR

Une application moderne peut combiner les deux : HTTP pour charger l'interface et une connexion persistante pour les événements en temps réel.

Méthodes d'attaque contre un cache web

Méthode	Principe
Web Cache Deception Web Cache Poisoning	Faire demander à la victime une URL trompeuse afin de stocker sa réponse privée. Influencer la réponse générée, puis faire mémoriser cette version malveillante pour d'autres visiteurs.
Entrée non incluse dans la clé	Exploiter un en-tête ou paramètre qui modifie la réponse sans distinguer les entrées de cache.
Divergence de normalisation	Profiter d'une différence de décodage ou de traitement du chemin entre deux composants.
Cache par extension	Ajouter une terminaison telle que .js pour faire classer une URL dynamique comme statique.
Cache buster	Ajouter une valeur unique au chemin ou aux paramètres pour éviter une entrée existante pendant les tests.

MÉTHODE DU TP : DECEPTION + DIVERGENCE DE CHEMIN + EXTENSION .JS

Outil / méthode	Utilité
Proxy d'interception	Observer les requêtes et réponses échangées par le navigateur.
HTTP history	Retrouver les échanges précédents et sélectionner une requête intéressante.
Burp Repeater	Modifier puis renvoyer manuellement une requête pour comparer les réponses.
Exploit Server	Héberger une page de laboratoire remise à la victime simulée.
Redirection JavaScript	Forcer le navigateur du laboratoire à charger l'URL forgée.
Test différentiel	Modifier un seul élément à la fois et observer le changement produit.
Test de répétition	Envoyer deux fois la même URL pour vérifier la transition MISS vers HIT.

OBSERVER



MODIFIER



RÉPÉTER



CONCLURE

PARTIE 1

Comprendre le mécanisme

À quoi sert un cache web ?



Le cache conserve temporairement une réponse et peut la réutiliser pour plusieurs requêtes.

Bénéfices

- ▶ temps de réponse réduit ;
- ▶ moins de charge serveur ;
- ▶ meilleure disponibilité.

Règle essentielle

Une réponse contenant des données privées ne doit jamais devenir une entrée de cache partagée.

Clé de cache simplifiée

hôte + chemin + paramètres retenus

- ▶ **MISS** : aucune copie disponible ; l'origine répond.
- ▶ **HIT** : une copie existe ; le cache la renvoie directement.
- ▶ **TTL / max-age** : durée pendant laquelle la copie reste valide.

```
HTTP/1.1 200 OK
Cache-Control: max-age=30
X-Cache: miss
Content-Type: text/html
```



même URL, moins de 30 s

```
HTTP/1.1 200 OK
Cache-Control: max-age=30
X-Cache: hit
```

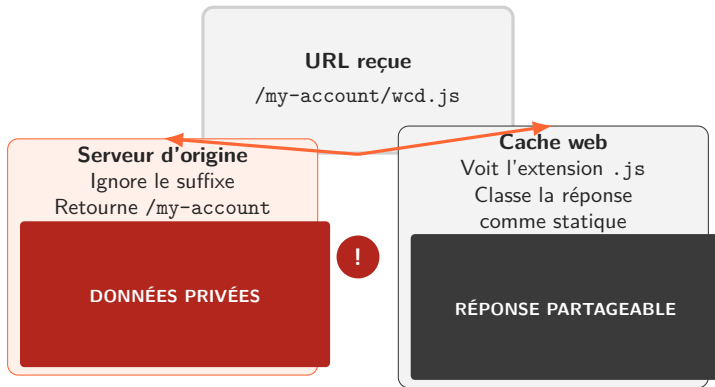
Deception ou Poisoning ?

	Web Cache Deception	Web Cache Poisoning
But	Faire stocker une réponse privée	Faire stocker une réponse malveillante
Action clé	La victime visite une URL trompeuse	L'attaquant influence la réponse mise en cache
Résultat	L'attaquant lit les données de la victime	Plusieurs utilisateurs reçoivent le contenu injecté
Exemple	/my-account/wcd.js	En-tête non inclus dans la clé de cache

Précision terminologique

Le laboratoire est parfois rangé dans la famille « attaques contre le cache web », mais le mécanisme présenté est précisément une **tromperie du cache web** (Web Cache Deception).

La cause racine : deux lectures incompatibles



Trois faiblesses doivent se combiner

1. Routage permissif

/my-account/abc renvoie encore la page du compte.

2. Cache par extension

La terminaison .js rend la réponse éligible au stockage.

3. Réponse mal protégée

La page authentifiée n'interdit pas explicitement le cache partagé.

ROUTAGE PERMISSIF

+

RÈGLE STATIQUE

+

PAS DE NO-STORE

=

FUITE DE DONNÉES

PARTIE 2

Démonstration dans Burp Suite

Environnement

- ▶ PortSwigger Web Security Academy ;
- ▶ Burp Proxy / HTTP history ;
- ▶ Burp Repeater ;
- ▶ Exploit Server ;
- ▶ compte attaquant : `wiener:peter` ;
- ▶ victime simulée : `carlos`.

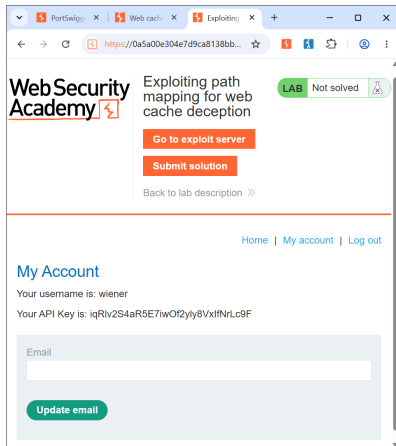
- 1 Observer la réponse authentifiée.
- 2 Tester un segment supplémentaire.
- 3 Ajouter une extension statique.
- 4 Vérifier MISS, puis HIT.
- 5 Faire visiter l'URL à la victime.
- 6 Relire la réponse depuis le cache.

Périmètre : laboratoire isolé et autorisé. Ne jamais reproduire cette procédure sur un système tiers sans autorisation écrite.

Observation

```
GET /my-account HTTP/1.1
Cookie: session=<WIENER>
```

Si cette réponse est partagée, le secret d'un utilisateur devient accessible à un autre.



Donnée ciblée

La clé API apparaît directement dans le HTML de la page personnalisée.

- ▶ réponse liée à une session ;
- ▶ secret réutilisable ;
- ▶ impact immédiat si la page est partagée.

NO-STORE OBLIGATOIRE

Étape 2 — Tester le routage permissif

```
GET /my-account/abc HTTP/1.1  
Host: YOUR-LAB-ID...  
Cookie: session=<WIENER>
```



200 OK

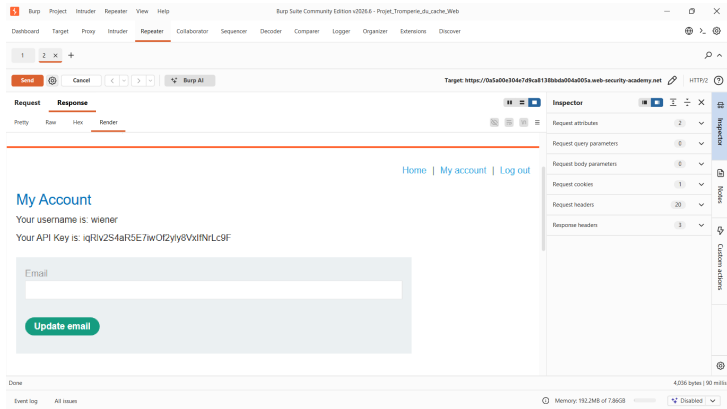
Interprétation

Le suffixe /abc n'est pas rejeté. L'origine continue à servir le contenu de /my-account.

Signal de vulnérabilité

Deux chemins différents donnent accès à la même ressource dynamique. Il faut maintenant vérifier si l'un de ces chemins trompe le cache.

Étape 3 — Ajouter une extension statique

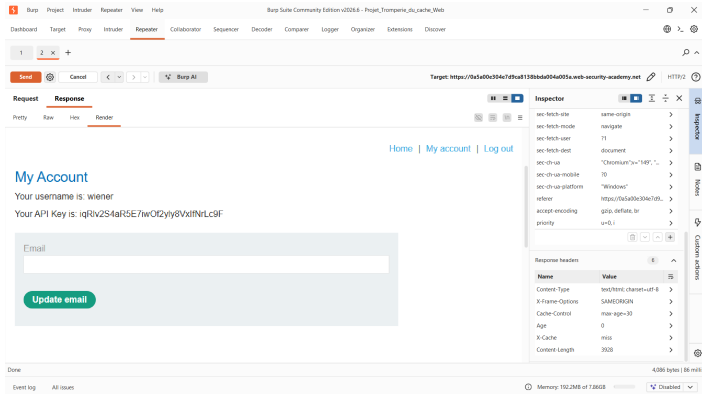


Modification

/my-account/
abc.js

- ▶ L'origine renvoie toujours la page du compte.
- ▶ Le cache reconnaît désormais une apparence « statique ».

Étape 4 — Prouver la mise en cache

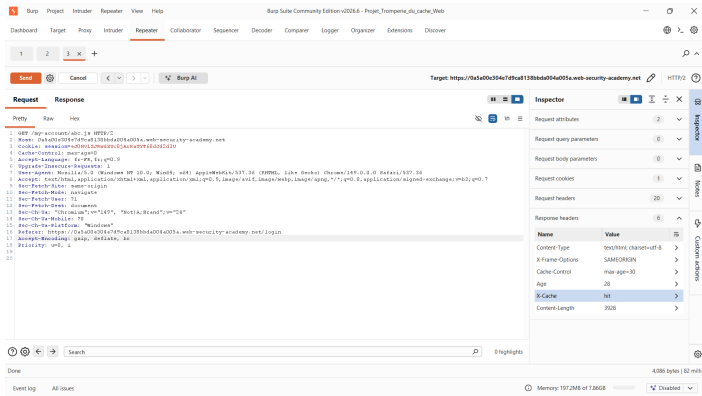


Cache-Control: max-age=30
X-Cache: miss

Premier accès

MISS : le cache n'avait pas encore la réponse. Il la récupère depuis l'origine et peut la stocker 30 secondes.

Étape 5 — Confirmer avec un HIT



Cache-Control: max-age=30
X-Cache: hit

Deuxième accès

HIT : la réponse privée est maintenant fournie depuis le cache.

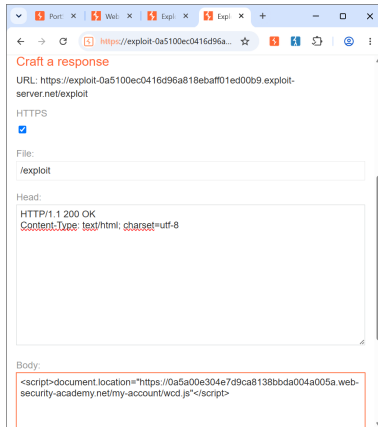
Vulnérabilité confirmée

La page de compte est devenue un objet de cache réutilisable.

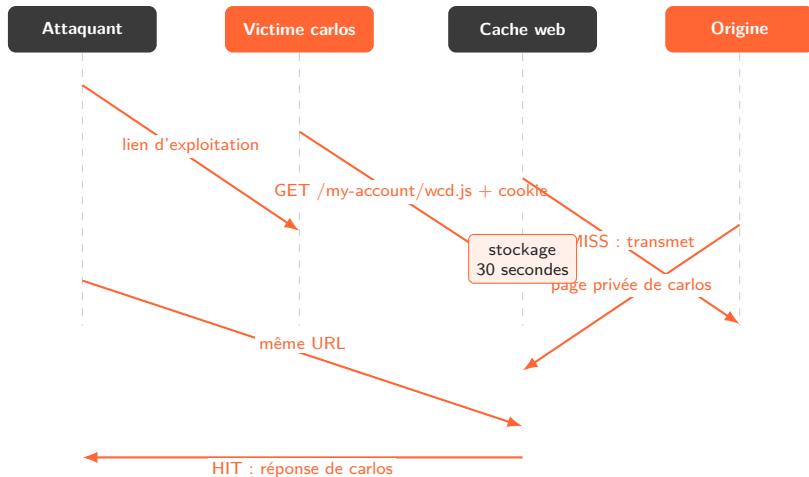
Étape 6 — Préparer la redirection

```
<script>
document.location =
  "https://YOUR-LAB-ID.web-security-
  academy.net/my-account/wcd.js";
</script>
```

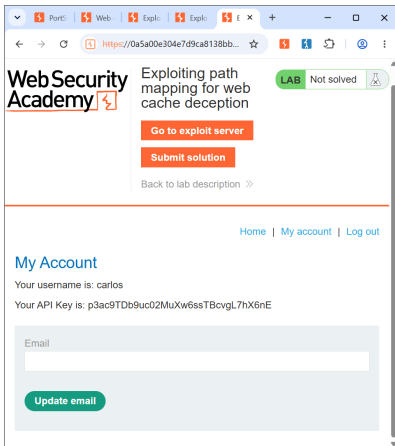
- ▶ URL unique pour éviter une ancienne entrée ;
- ▶ la victime envoie automatiquement son cookie ;
- ▶ le suffixe .js déclenche la règle de cache.



Étape 7 — Ce qui se passe après le clic



Résultat — La réponse de carlos est exposée



Preuve d'impact

La page mise en cache révèle :

- ▶ l'identité de carlos ;
- ▶ sa clé API ;
- ▶ une réponse obtenue sans sa session.

Bonne pratique de rapport

Sur un système réel : masquer le secret, le révoquer et le transmettre par canal sécurisé.

PARTIE 3

Impact, détection et correction

Côté application

- ▶ route dynamique tolérant un suffixe ;
- ▶ cache fondé sur une extension ;
- ▶ réponse authentifiée stockable ;
- ▶ clé de cache ne séparant pas les sessions.

Côté attaque

- ▶ victime connectée ;
- ▶ visite de l'URL forgée ;
- ▶ chemin unique ou non encore rempli ;
- ▶ récupération avant expiration du TTL.

UNE SEULE CONDITION CASSÉE PEUT INTERROMPRE LA CHAÎNE

Critère	Évaluation
Confidentialité	Élevée : secrets et données de compte
Intégrité	Indirecte : actions possibles avec une clé volée
Disponibilité	Faible dans ce scénario
Complexité	Faible à modérée après détection du chemin
Interaction	La victime doit charger le lien



Dans les journaux

- ▶ chemins dynamiques finissant par .js, .css, .png ;
- ▶ nombreux suffixes aléatoires ;
- ▶ même URL demandée par plusieurs sessions ou IP ;
- ▶ accès rapprochés dans la fenêtre du TTL.

Dans les réponses

- ▶ X-Cache: hit sur un espace authentifié ;
- ▶ Cache-Control: public ou max-age inattendu ;
- ▶ contenu HTML privé sur une URL d'apparence statique ;
- ▶ données d'un autre compte après répétition.

Alerte prioritaire

Un HIT sur une page de compte doit être considéré comme un incident potentiel jusqu'à preuve du contraire.

- ❶ Interdire le stockage des pages privées.
- ❷ Retourner 404 pour les chemins non canoniques.
- ❸ Mettre en cache uniquement des routes publiques explicitement déclarées.
- ❹ Aligner la normalisation entre CDN, proxy et origine.
- ❺ Séparer les ressources sous un préfixe dédié, par exemple `/assets/`.

```
Cache-Control: no-store, private  
Pragma: no-cache
```

Défense en profondeur

Ne pas stocker une réponse associée à un cookie de session ou à un en-tête `Authorization`.

Attention

Ajouter le cookie à la clé de cache limite le partage, mais ne remplace pas `no-store` pour les secrets.

Test	Résultat attendu
GET /my-account/abc.js	404, redirection sûre ou aucune donnée privée
Deux requêtes identiques	Aucun X-Cache: hit sur la page authentifiée
Requête sans session	Redirection vers la connexion
Inspection des en-têtes	Cache-Control: no-store, private
Test avec deux comptes	Aucune donnée croisée entre utilisateurs
Variantes encodées	Même interprétation par le cache et l'origine

CORRIGER



PURGER LE CACHE



ROTATION DES SECRETS



RETESTER

1. Comprendre

Le cache et l'origine peuvent interpréter différemment une même URL.

2. Vérifier

Une transition MISS vers HIT prouve la réutilisation de la réponse.

3. Protéger

Les réponses authentifiées doivent être non stockables et les routes strictement canoniques.

La performance ne doit jamais transformer une réponse privée en contenu public.

TROMPERIE DU CACHE WEB

Questions ?

Démonstration réalisée dans un environnement de laboratoire autorisé.

Gana Faye | Sécurité des applications web